

Allow main function in named modules

Document #: P3422R1
Date: 2024-11-22
Project: Programming Language C++
Audience: Core Language Evolution Group
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

§ 1 Introduction

This paper proposes to allow using the main function in named module.

§ 2 Motivation

The motivation example is for test. Considering:

```
export module A;  
class non_exported { ... };  
export template <class C>  
class exported {  
    non_exported ...;  
    ...  
};
```

Then it is a question for how to write the unit tests for non_exported.

The most straightforward solution is to write the unit tests in the same module so that we can have the full visibility for the module. e.g.,

```
module A; // or `module A:test_non_exported;` to make it more expressive.  
int main() {  
    non_exported ...;  
    ...  
}
```

However, the above code is illegal according to the current wording.

```
[basic.start.main]p3:  
A program that declares:  
- ...  
- a function main that belongs to the global scope and is attached to a  
  named module, or  
- ...  
is ill-formed.
```

If we want to use [language linkage](#) to attach the main function to the global module, we can't do that neither according to [\[basic.start.main\]p3](#) too:

```
The main function shall not be declared within a linkage-specification
([dcl.link]).
```

And also if the whole program is made of modules, it's annoying to have a single-non-module file just for main.

So we propose allowing the use of a main function in named modules to make the above use cases and other use cases easier and more straightforward.

BTW, both Clang and GCC didn't implement [\[basic.start.main\]p3.2](#) and somehow gives the main the function the “correct” linkage: <https://godbolt.org/z/5Pfjzf87P> . And if we accept the proposal, we may only need to do some conforming implementations. e.g., in clang, now the main function in the above example is attached to the named module and we should attach it to the global module: <https://godbolt.org/z/6YMTdaPaP>

§ 3 Proposed behavior

The main function in the global scope should always be attached to the global module.

§ 4 Design Decisions

There was a paper [P1203R0](#) that attempts to do something similar to this paper.

The only difference is that P1203R0 proposes to allow attaching the main function to named modules, while this paper proposes to always attach the main function to the global module.

Another difference is that P1203R0 seeks to introduce a new feature called `exported main()`, which stipulates that: (1) if a module has an exported `main()`, that serves as the main function of the program. (2) if the program lacks an exported `main()` but has (potentially multiple) non-exported main function, one can be chosen at link time (e.g., as a driver for a test suite), which reflects the existing behavior.

While this approach looks promising, we realize that it may be challenging to implement in practice.

On the one hand, the ODR issue is **already** forbidden clearly in [\[basic.def.odr\]](#). The toolchains have just failed to enforce this rigorously.

On the other hand, currently, if there are multiple main functions during linking, the linker will select the first main function. Although this is a hack, it is actually used in practice. We have utilized this internally, and according to the minutes of P1203R0, Bloomberg has done so as well. Thus, it is a technique to hijack the main function, even if it is considered

a hack. The challenge here is that we will need more effort to implement (1) to instruct the linker to fetch the exported `main()` instead of the first `main` function.

So, while it initially seems better, we have some concerns: (1) it changes an assumption of the ecosystem: we can't hijack the `main` function by altering the linking order (which is a hack but is used in practice). (2) it requires more implementation effort. From the authors' experience, resources to implement new features are limited. This is particularly true when it requires vendors to modify the ABI and linkers, while most developers involved are frontend developers. The cost of development in other areas is quite high, and we believe vendors can better allocate their time elsewhere.

We hope this will be a small change that benefits people while preserving the existing mechanism for loading programs. Thus, we have aimed to keep our approach as simple as possible, similar to how we handle allocation and deallocation functions, by always attaching the `main` function to the global module.

§ 5 Impacts

§ 5.1 Impact on users

Since both Clang and GCC somehow perform the desired behavior, at least the users of Clang and GCC won't be affected. We didn't check the behavior of MSVC, but we believe it should have the same impact as the standard didn't allow it.

§ 5.2 Impact on standard

Given this change asks we allow something forbidden before, it shouldn't introduce any regression changes.

§ 5.3 Impact on implementation

We believe it is easy to change the modules ownership for the specific `main` function.

And given both Clang and GCC do the wanted behavior, we only need to do some conforming change in Clang and GCC.

§ 5.4 Summary

Given that we didn't see any negative impact and considering the scale of the change, we suggest applying this in C++20 or recommending that vendors implement it in C++20.

§ 6 Proposed Wording

Modify p3 of §6.9.3.1 [`basic.start.main`] :

A program that declares

- a variable `main` that belongs to the global scope, or
- ~~a function `main` that belongs to the global scope and is attached to a named module, or~~
- a function template `main` that belongs to the global scope, or
- an entity named `main` with C language linkage (in any namespace)

is ill-formed. The name `main` is not otherwise reserved.

Modify p7.2 of §10.1 [**module.unit**]:

Otherwise, if the declaration

- is a namespace-definition with external linkage , or
- appears within a linkage-specification ([`dcl.link`]), or
- is a function named `main` that belongs to the global scope

it is attached to the global module.